

AUTOMATION & CONFIG MANAGEMENT

Ansible

Top 50 Interview Questions

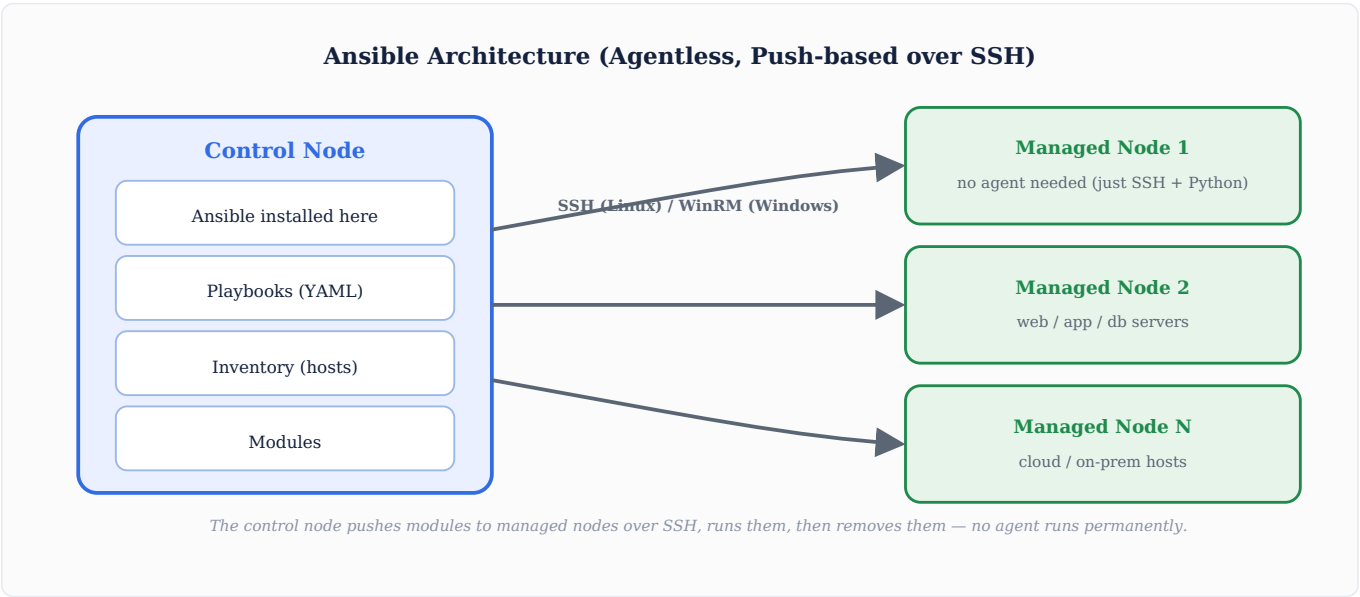
For DevOps Engineers — Detailed Answers
With essential commands & code snippets

Covers Architecture & Agentless model • Inventory • Playbooks/Plays/Tasks
Modules, Variables & Jinja2 • Roles • Vault • Handlers • CI/CD

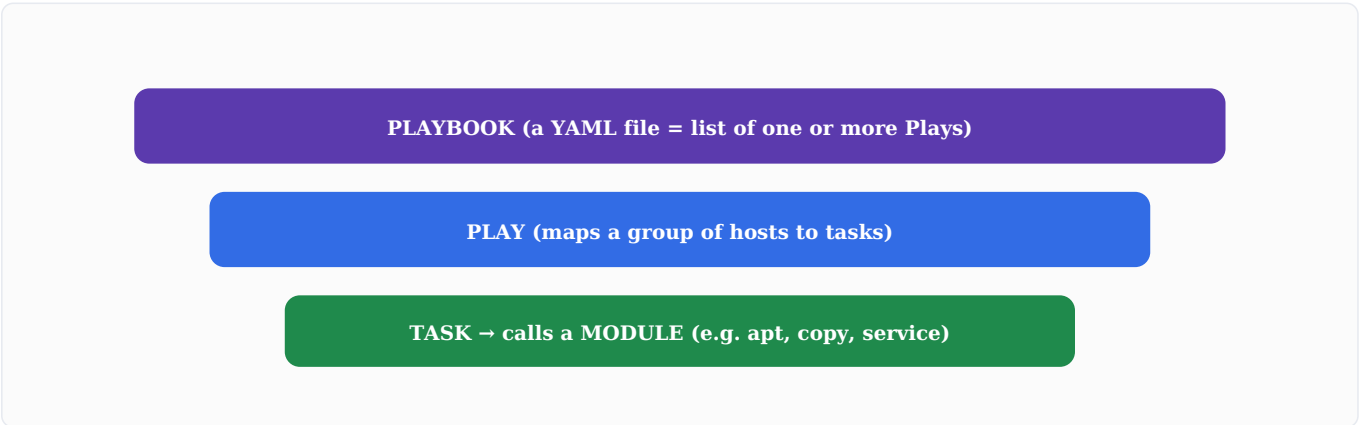
Quick Visual Primer

Two ideas underpin most Ansible interviews: the **agentless, push-based architecture** and the **Playbook → Play → Task → Module** hierarchy.

Ansible Architecture



Playbook Hierarchy



Mental model: A **Playbook** is a YAML file made of **Plays**; each Play targets hosts and runs **Tasks**; each Task calls a **Module** that does the actual work — idempotently.

Essential Ansible Commands

Command	What it does
<code>ansible --version</code>	Show installed version and config
<code>ansible all -m ping</code>	Test connectivity to all hosts (ad-hoc)
<code>ansible web -m apt -a "name=nginx state=present" --become</code>	Ad-hoc: install a package
<code>ansible-playbook site.yml</code>	Run a playbook
<code>ansible-playbook site.yml --check --diff</code>	Dry run, showing changes
<code>ansible-playbook site.yml --tags "install"</code>	Run only tagged tasks
<code>ansible-inventory --list</code>	Show the parsed inventory
<code>ansible host -m setup</code>	Gather and display facts
<code>ansible-galaxy init myrole</code>	Scaffold a new role structure

```
ansible-vault encrypt secrets.yml
```

Encrypt a secrets file

Top 50 Ansible Interview Questions & Answers

Grouped by topic, from fundamentals to CI/CD, with commands and code where useful. Answers are concise but keep the technical terms; **bolded** words and `commands/code` are worth memorizing.

Fundamentals & Architecture

Q1. What is Ansible?

Answer: **Ansible** is an open-source **automation tool** (by Red Hat) used for **configuration management, application deployment, and orchestration**. It is **agentless**, uses **SSH** to connect to managed nodes, and uses simple **YAML** files (playbooks) to describe automation tasks declaratively.

Q2. What are the main use cases of Ansible?

Answer: **Configuration management** (enforcing consistent system state), **application deployment, provisioning, orchestration** (coordinating multi-tier rollouts), and **continuous delivery** within CI/CD pipelines.

Q3. What does it mean that Ansible is 'agentless'?

Answer: **Agentless** means you **don't install any Ansible software/daemon on the managed nodes**. Ansible connects over **SSH** (or WinRM for Windows), pushes the required modules, runs them, and removes them. This reduces overhead, security surface, and maintenance.

Q4. What language are Ansible playbooks written in?

Answer: Playbooks are written in **YAML** (YAML Ain't Markup Language) — a human-readable data format that relies on **indentation** (spaces, not tabs) to define structure.

Q5. What is the difference between Ansible and tools like Puppet/Chef?

Answer: **Ansible** is **agentless** (push-based over SSH) and uses **YAML**, making it simple to start. **Puppet/Chef** are typically **agent-based (pull model)** and use their own DSLs (Ruby-based). Ansible's agentless, low-setup approach is a key differentiator.

Q6. What is the difference between Ansible (Core) and Ansible Tower/AWX?

Answer: **Ansible Core** is the open-source command-line engine. **Ansible Tower** (commercial) / **AWX** (open-source upstream) add a **web UI, REST API, RBAC, scheduling, logging, and a dashboard** for managing Ansible at enterprise scale.

Q7. What are the key benefits of Ansible?

Answer: **Agentless** (no software on nodes), **simple YAML** syntax, **idempotent** operations, **push-based** execution, a huge library of **modules, reusable roles**, and easy integration with CI/CD — making it approachable yet powerful.

Q8. What is idempotency in Ansible and why does it matter?

Answer: **Idempotency** means running the same playbook multiple times produces the **same end state** without making changes if the system is already in the desired state. Most Ansible modules are idempotent — e.g. installing a package that's already present reports **'ok'**, not **'changed'**. This makes runs safe and predictable.

Q9. What are the main components of Ansible?

Answer: **Control node** (where Ansible runs), **Managed nodes** (the target hosts), **Inventory** (list of hosts), **Modules** (units of work), **Tasks** (calls to modules), **Playbooks** (YAML automation files), **Roles** (reusable structures), and **Plugins**.

Q10. How do you check the installed Ansible version?

Answer: Run `ansible --version`. It shows the Ansible version, the config file in use, module/library paths, and the Python version it runs on.

Inventory & Hosts

Q11. What is an Ansible inventory?

Answer: The **inventory** is a file (default `/etc/ansible/hosts`) that **lists the managed nodes** Ansible controls, organized into **groups**. It can be in **INI** or **YAML** format and defines which hosts tasks run against.

Q12. Show an example of a simple INI inventory.

Answer: `[web]`

`web1.example.com`

`web2.example.com`

`[db]`

`db1.example.com ansible_host=10.0.0.5`

Here `web` and `db` are groups, and `ansible_host` sets the connection IP.

Q13. What is the difference between a static and a dynamic inventory?

Answer: A **static inventory** is a manually maintained file listing hosts. A **dynamic inventory** is generated by a **script or plugin** that pulls hosts in real time from sources like **AWS, Azure, or GCP** — essential for cloud environments where hosts change frequently.

Q14. What are inventory groups and why are they useful?

Answer: **Groups** let you organize hosts (e.g. `[web]`, `[db]`) so you can target a set of machines at once. A special group `[all]` includes every host. You can also nest groups with `[parent:children]`.

Q15. What are host variables and group variables?

Answer: **Host variables** apply to a single host; **group variables** apply to all hosts in a group. They can be defined inline, or in `host_vars/` and `group_vars/` directories. They let you customize behavior per host/group (e.g. different ports).

Q16. How do you list all hosts in the inventory?

Answer: `ansible all --list-hosts` lists every host. `ansible web --list-hosts` lists hosts in the `web` group. `ansible-inventory --list` shows the full structured inventory.

Q17. What is the 'ansible_host' and 'ansible_user' variable?

Answer: `ansible_host` sets the actual address/IP Ansible connects to (when it differs from the inventory name). `ansible_user` sets the remote SSH user. They are **connection (behavioral) variables** that control how Ansible reaches a node.

Playbooks, Plays, Tasks & Handlers

Q18. What is an Ansible playbook?

Answer: A **playbook** is a **YAML file containing one or more plays** that define the automation to run on managed nodes. It is the primary way to orchestrate complex, ordered, multi-step configuration in a repeatable, declarative manner.

Q19. What is a play?

Answer: A **play** is a single unit in a playbook that **maps a group of hosts to a set of tasks**. It specifies the target hosts (`hosts:`), connection/privilege settings, variables, and the tasks to execute on those hosts.

Q20. What is a task?

Answer: A **task** is a **single action** within a play that calls a **module** (e.g. install a package, copy a file). Tasks run **in order, top to bottom**, on all targeted hosts.

Q21. Show the structure of a basic playbook.

Answer: `---`

`- name: Install and start nginx`

`hosts: web`

`become: yes`

`tasks:`

`- name: Install nginx`

`apt:`

`name: nginx`

`state: present`

Q22. How do you run a playbook?

Answer: `ansible-playbook playbook.yml` runs it. Useful flags: `-i inventory` (specify inventory), `--check` (dry run), `--limit web1` (restrict hosts), `--tags / --skip-tags`, and `-v` (verbose).

Q23. What does the 'become' keyword do?

Answer: `become` enables **privilege escalation** — running tasks as another user, typically **root** via `sudo`. `become: yes` runs the task with elevated rights; `become_user` sets which user to become.

Q24. What is an ad-hoc command in Ansible?

Answer: An **ad-hoc command** runs a **single task quickly without writing a playbook**. Example: `ansible all -m ping` pings all hosts, or `ansible web -m apt -a "name=nginx state=present" --become` installs nginx. Good for one-off actions.

Q25. What is the 'ping' module and how is it different from a network ping?

Answer: The Ansible `ping` module (`ansible all -m ping`) **tests connectivity AND that Python is usable** on the managed node, returning `pong` on success. Unlike ICMP `ping`, it actually verifies Ansible can connect and run modules via SSH.

Q26. What is a handler and when does it run?

Answer: A **handler** is a special task that runs **only when notified by another task** that reported a **change**, and it runs **once at the end of the play**. Typical use: **restart a service** after its config file changes (a task uses `notify: Restart nginx`).

Q27. What is the difference between a task and a handler?

Answer: A **task** runs **every time** in sequence. A **handler** runs **only if triggered by a `notify`** from a changed task, and only once at the play's end. Handlers are used for actions that should happen only when something actually changed.

Modules, Variables & Templates

Q28. What is an Ansible module?

Answer: A **module** is a **reusable, standalone unit of code** that performs a specific task (e.g. `apt`, `yum`, `copy`, `service`, `file`, `user`). Tasks invoke modules. Ansible ships with thousands of built-in modules; most are **idempotent**.

Q29. Name some commonly used Ansible modules.

Answer: `apt` / `yum` / `dnf` (packages), `copy` / `template` / `file` (files), `service` / `systemd` (services), `user` / `group` (accounts), `command` / `shell` (run commands), `git`, and `lineinfile` (edit lines in files).

Q30. What is the difference between the 'command' and 'shell' modules?

Answer: `command` runs a command **without a shell**, so shell features like pipes (`|`), redirection (`>`), and variables don't work — but it's safer. `shell` runs through a **shell**, so those features work. Prefer `command` unless you specifically need shell functionality.

Q31. What are Ansible variables and how do you define them?

Answer: **Variables** store reusable values to make playbooks flexible. They can be defined in **playbooks** (`vars:`), **inventory**, **group_vars/host_vars** files, **included var files** (`vars_files`), or passed at runtime with `--extra-vars`. Reference them with `{{ variable_name }}` (Jinja2).

Q32. What is Jinja2 in Ansible?

Answer: **Jinja2** is the **templating engine** Ansible uses for variable substitution, expressions, filters, and conditionals — written as `{{ }}` for variables and `{% %}` for logic. It powers dynamic values and the `template` module.

Q33. What is the difference between the 'copy' and 'template' modules?

Answer: `copy` transfers a file **as-is** to the managed node. `template` processes a **Jinja2 template (.j2)** first — substituting variables — then copies the rendered result. Use `template` when the file content must be dynamic (e.g. config with host-specific values).

Q34. What are Ansible facts?

Answer: **Facts** are **system information automatically gathered** from managed nodes (OS, IP, memory, CPU, etc.) at the start of a play via the `setup` module. You reference them like `{{ ansible_facts['os_family'] }}`. Run `ansible host -m setup` to see all facts.

Q35. What is the 'register' keyword used for?

Answer: **register captures the output/result of a task** into a variable, which you can then use later (e.g. in a condition or debug). Example: `register: result`, then `when: result.rc != 0`.

Q36. How do you use conditionals in Ansible?

Answer: Use the `when` keyword on a task to run it only if a condition is true. Example: `when: ansible_facts['os_family'] == "Debian"` runs the task only on Debian-based systems.

Q37. How do you loop over items in a task?

Answer: Use the `loop` keyword (modern) — e.g. install several packages:

```
- apt:
  name: "{{ item }}"
  state: present
  loop:
    - nginx
    - git
```

(`with_items` is the older equivalent.)

Roles, Vault & Advanced Topics

Q38. What is an Ansible role?

Answer: A **role** is a **standardized way to organize and reuse** related tasks, variables, files, templates, and handlers into a structured directory. Roles make playbooks **modular, shareable, and clean** — instead of one huge file, you compose roles.

Q39. What is the standard directory structure of a role?

Answer: A role has standard folders: `tasks/` (main tasks), `handlers/`, `templates/` (Jinja2), `files/` (static files), `vars/` and `defaults/` (variables), `meta/` (dependencies). Each has a `main.yml` entry point. Generate one with `ansible-galaxy init rolename`.

Q40. What is the difference between 'vars' and 'defaults' in a role?

Answer: `defaults/` holds the **lowest-priority default variables** (easily overridden). `vars/` holds **higher-priority variables** that are harder to override. Use `defaults` for sensible defaults users can change, and `vars` for values that should generally stay fixed.

Q41. What is Ansible Galaxy?

Answer: **Ansible Galaxy** is a **public repository for sharing and downloading roles and collections**. Use `ansible-galaxy install username.rolename` to download a community role, or `ansible-galaxy init` to scaffold a new role's structure.

Q42. What is an Ansible collection?

Answer: A **collection** is a **packaging format that bundles related roles, modules, plugins, and playbooks** together for easy distribution and versioning. Collections are installed from Galaxy/Automation Hub (e.g. `ansible-galaxy collection install community.general`).

Q43. What is Ansible Vault?

Answer: **Ansible Vault encrypts sensitive data** (passwords, keys, secrets) within files so they can be safely stored in version control. Create/edit with `ansible-vault create secrets.yml` or `ansible-vault encrypt file.yml`, and run with `--ask-vault-pass` or a vault password file.

Q44. How do you encrypt and decrypt a file with Ansible Vault?

Answer: `ansible-vault encrypt file.yml` encrypts; `ansible-vault decrypt file.yml` decrypts; `ansible-vault edit file.yml` edits in place; `ansible-vault view file.yml` displays it. Run a playbook using vaulted data with `ansible-playbook site.yml --ask-vault-pass`.

Q45. What are tags in Ansible?

Answer: **Tags** let you **label tasks/roles** so you can run or skip specific parts of a playbook. Example: run only tagged tasks with `ansible-playbook site.yml --tags "install"`, or skip with `--skip-tags`. Useful for large playbooks.

Q46. What is the difference between 'include_tasks' and 'import_tasks'?

Answer: **import_tasks** is **static** — tasks are included at **parse time** (before execution). **include_tasks** is **dynamic** — included at **runtime**, so it can be used with loops and conditionals that depend on runtime data. Static is processed upfront; dynamic is evaluated during the run.

Q47. What does the '--check' mode do?

Answer: `--check` runs the playbook in '**dry run**' (**check**) **mode** — it reports what **would change without actually making changes**. Combined with `--diff`, it shows the differences too. Great for safely previewing impact.

Q48. How do you handle errors in Ansible (ignore_errors, failed_when, blocks)?

Answer: Use `ignore_errors: yes` to continue despite a task failure, `failed_when` to define custom failure conditions, and **blocks** with `rescue` and `always` sections (like try/catch/finally) for structured error handling.

Q49. What is the difference between a static and dynamic include of roles/tasks (and when to use each)?

Answer: **Static** (`import_*`) is resolved before the play runs — better for predictability and works with `--list-tasks`. **Dynamic** (`include_*`) is resolved during the run — needed when the inclusion depends on **runtime variables, loops, or conditionals**.

Q50. How does Ansible fit into a CI/CD pipeline?

Answer: In CI/CD (Jenkins, GitLab CI, GitHub Actions), Ansible is the **configuration/deployment step**: after infrastructure is provisioned (often by Terraform), the pipeline runs `ansible-playbook` to **configure servers and deploy the application**. Vault secures secrets, and idempotency makes repeated runs safe.
